CO2 Assignment 2 – Complete Solutions

C programming answers, explanations, sample calculations, test evidence and complexity analysis.

Important assumptions: Where the assignment does not specify variable types or sample passenger data, this solution states the assumptions used. The C examples use standard C and include input validation where appropriate.

Q1 – Precedence, Associativity and the Discount Bug

1. Prediction and operator order

Assume **int** variables for price, qty, senior, total and net; senior is 1 for an eligible senior citizen and 0 otherwise. Multiplication and division have equal precedence and are evaluated left-to-right; addition and subtraction also have equal precedence and are evaluated left-to-right. Multiplication/division happen before addition/subtraction.

For the given expression, the effective grouping is: `((price*qty) - (((price*qty)*15)/100*senior) + (((price*qty)*5)/100))`. With integer operands, each division truncates its fractional part.

Bill	price × qty	senior	discount term	GST term	total	net
A	200 (100×2)	1	30	10	180	130
B	199 (199×1)	1	29	9	179	179
C	500 (250×2)	0	0	25	525	525

Net for A is 180 – 50 = 130 because total > 1000 is false. B and C also do not exceed 1000. To demonstrate the net discount threshold, a gross bill producing total above 1000 would have Rs 50 subtracted.

2. What is wrong, and the corrected design

• The original GST is calculated on the original gross amount, not on the amount after the senior discount. If GST is intended on the discounted subtotal, this is a logic error.

• Integer division loses fractional paise: for gross 199, 199×15/100 becomes 29 and 199×5/100 becomes 9. Use double for percentage calculations and round only when producing the payable currency amount.

• The expression count = count++ + 1 modifies and also reads count without a sequencing point between those operations. Its behaviour is undefined in C. Replace it with count++;

```
#include <stdio.h>
int main(void) {
    double price = 100.0;
    int qty = 2, senior = 1, count = 0;
    double gross = price * qty;
    double discount = senior ? gross * 0.15 : 0.0;
    double taxable = gross - discount;
    double gst = taxable * 0.05;
    double total = taxable + gst;
    double net = (total > 1000.0) ? total - 50.0 : total;

    count++;
    printf("Gross = %.2f\nDiscount = %.2f\nGST = %.2f\n", gross, discount, gst);
    printf("Total = %.2f\nNet = %.2f\nCount = %d\n", total, net, count);
    return 0;
}
```

For the example above: gross = Rs 200.00, discount = Rs 30.00, taxable = Rs 170.00, GST = Rs 8.50, total/net = Rs 178.50. If the pharmacy's policy instead charges GST on the full price, calculate GST from gross explicitly and document that policy.

3. Alternative with intermediate variables and complexity

The code above uses both meaningful intermediate variables and clear parentheses where needed. It is easier to test each component and reduces precedence mistakes. It performs a constant number of arithmetic operations: O(1) time. A few temporary scalar variables use O(1) extra space. A compiler may keep them in registers; readability and correctness improve without changing the asymptotic complexity.

Compiler warning: compile with `gcc -Wall -Wextra -std=c11 file.c`. GCC commonly warns about the unsequenced modification/read in the count expression (often described as an operation on ‘count’ that may be undefined). A warning is not a runtime fix; the expression must be rewritten.

Q2 – When Does Short-Circuit Evaluation Matter?

1. Demonstration

Assume the concession rule is `(senior && eligible_class) || verify_disability_card(id)`, and `verify_disability_card` is the slow server call. In C, && stops when its left operand is false; || stops when its left operand is true. Both operators evaluate operands left-to-right.

```
#include <stdio.h>
int calls = 0;
int verify_disability_card(int id) {
    calls++;
    /* A real program would contact the server. */
    return id % 10 == 0;   /* sample result only */
}
int concession(int senior, int eligible, int id) {
    return (senior && eligible) || verify_disability_card(id);
}
int main(void) {
    int result = concession(1, 1, 123);
    printf("Concession=%d, server calls=%d\n", result, calls);
    return 0;
}
```

Here senior=1 and eligible=1, so the left side of || is true and the server function is not called. If senior=0, the right side of && is skipped; the OR then evaluates the verification function.

2. Call-count experiment for 1,000 passengers

Use a stated sample distribution: 1,000 passengers; 300 are seniors; among those 300, 240 have an eligible ticket class and 60 do not. Thus the first condition is true for 240 passengers. The expression in the order shown calls the server for the other 760 passengers.

Condition order	Server calls	Reason
(senior && eligible) || verify(card)	760	Skip verification for 240 already-qualified passengers.
verify(card) || (senior && eligible)	1,000	The first operand is evaluated for every passenger.
(senior & eligible) | verify(card)	1,000	Bitwise operators do not short-circuit; both sides are evaluated.

The counts depend on the actual passenger distribution. In general, with n passengers and q passengers satisfying senior && eligible, the first ordering makes n–q verification calls. The reversed ordering makes n calls.

3. Cheap/likely-false condition and audit-log change

Put a cheap condition first when it can decide the result and avoid an expensive call. For &&, a cheap condition likely to be false is useful first. For ||, a cheap condition likely to be true is useful first. The order is irrelevant to the number of calls if the first operand always forces evaluation of the second, or if the second operation is equally cheap. Do not reorder expressions when operands have required side effects or when doing so changes the intended logic.

If the verification function must create an audit record for every passenger, it must be called for all 1,000 passengers. Call it explicitly once per passenger, store its result, and then evaluate the concession rule. This preserves the audit requirement; short-circuit savings are no longer the priority.

```
int verified = verify_disability_card(id); /* exactly one audit/check call */
int allowed = (senior && eligible_class) || verified;
```

For n passengers: short-circuit version shown above uses n–q slow calls; reversed order or non-short-circuit & and | uses n calls. Explicit auditing also uses n calls. Each passenger's remaining Boolean work is O(1), so total time is O(n); the call count is the important difference.

Q3 – Engineer a Device-Status Register with Bitwise Operators

1. Completed functions

```
#include <stdio.h>
enum { LIGHT=0, FAN=1, AC=2, HEATER=3, PUMP=4 };

unsigned char switch_on(unsigned char reg, int dev) {
    return (unsigned char)(reg | (1u << dev));
}
unsigned char switch_off(unsigned char reg, int dev) {
    return (unsigned char)(reg & ~(1u << dev));
}
```

```
unsigned char toggle(unsigned char reg, int dev) {
    return (unsigned char)(reg ^ (1u << dev));
}
int is_on(unsigned char reg, int dev) {
    return (reg & (1u << dev)) != 0;
}
void show(unsigned char reg) {
    for (int bit=7; bit>=0; bit--)
        putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}
int main(void) {
    unsigned char reg=0;
    reg=switch_on(reg,LIGHT); /* 00000001 = 1 */
    reg=switch_on(reg,AC);    /* 00000101 = 5 */
    show(reg);
    reg=toggle(reg,FAN);      /* 00000111 = 7 */
    reg=switch_off(reg,LIGHT);/* 00000110 = 6 */
    show(reg);
    printf("AC is %s, PUMP is %s\n",
        is_on(reg,AC)?"ON":"OFF", is_on(reg,PUMP)?"ON":"OFF");
    return 0;
}
```

Expected output:

```
00000101
00000110
AC is ON, PUMP is OFF
```

2. Explanation and alternative array

`reg & (1u << dev)` first creates a mask at bit dev, then tests that bit. `(reg & 1) << dev` first keeps only bit 0, then shifts that bit; it does not test the original bit dev. For example, if reg=4 (binary 00000100) and dev=2, the first expression is nonzero, while the second is zero.

Use unsigned char so bit 7 is handled as an unsigned bit pattern. With signed char, setting the top bit can produce a value not representable as signed char; conversions and right shifts of negative values can be implementation-dependent. Use unsigned operands such as 1u for masks.

With `char state[8]`, set/off/toggle/test use direct indexing and are O(1). It uses 8 bytes per home versus 1 byte for the packed register: for 1,000,000 homes, about 8,000,000 bytes versus 1,000,000 bytes (ignoring other storage overhead). Counting ON devices scans all 8 entries in either design, O(8)=O(1) per home. The bit version can count set bits using a loop or a compiler bit-count function.

Q4 – else–if Ladder or switch?

1. Two implementations

```
char grade_if(int marks) {
    if (marks < 0 || marks > 100) return '?';
    if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}

char grade_switch(int marks) {
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {
        case 10: case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}
```

Test results:

Marks	else–if	switch	Explanation
100	S	S	Valid top mark
90	S	S	S boundary

Marks	else–if	switch	Explanation
89	A	A	A upper end
0	F	F	Valid lowest mark
-1	?	?	Rejected
101	?	?	Rejected

A stronger boundary test set checks one mark below, at and above every boundary: –1, 0, 1, 49, 50, 51, 59, 60, 61, 69, 70, 71, 79, 80, 81, 89, 90, 91, 99, 100, 101.

2. switch fall-through and design choice

A missing break or return causes execution to continue into the next case (fall-through). In this function, return ends the function, so each case returns immediately. If using assignments instead, add break after each case. A switch compares one integral expression with constant case values; it cannot directly express a range such as 75–84. The marks/10 mapping converts ranges into integer cases.

The else–if ladder is easiest when boundaries are ranges or may change independently. The switch is clear when using marks/10. A lookup table is compact and O(1), but requires careful bounds checking. For a scholarship rule depending on grade and income, a small lookup table indexed by grade and income band is suitable if the policy is a fixed matrix; nested if statements are clearer if the rules are few and irregular.

```
char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
char g;
if (marks < 0 || marks > 100) g='?';
else g=grades[marks/10];
```

The table uses 11 bytes and one indexed lookup, O(1) time. The ladder uses up to k comparisons for k grade bands (also O(1) here because k=6, but O(k) in a generalized ladder).

Q5 – Count the Work Done by a Loop

1. Improved prime test

```
#include <stdio.h>
unsigned long checks = 0;

int is_prime(unsigned long n) {
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i=2; i <= n/i; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}
int main(void) {
    unsigned long tickets[] = {97,100,7919,1000003};
    for (int k=0;k<4;k++) {
        int p=is_prime(tickets[k]);
        printf("%8lu %-9s divisions = %lu\n",
            tickets[k], p?"prime":"not prime", checks);
    }
    return 0;
}
```

Using i <= n/i avoids the possible overflow in i*i. The loop tests divisors only up to the square root. It increments checks once for each modulo/divisibility test.

```
     97 prime     divisions = 8
    100 not prime divisions = 1
   7919 prime     divisions = 87
1000003 prime    divisions = 999
```

2. Loop versions, edge cases and complexity

Equivalent improved loops can use while or do–while. For while: initialise i=2; while (i <= n/i) { checks++; if (n%i==0) return 0; i++; }. For do–while, first handle n<2, then perform one test and continue while i<=n/i; however n=2 and n=3 have no divisor to test, so a do–while would incorrectly test once unless a pre-check handles n<4.

If the condition is changed to i < n, the program may do far more tests: a prime n requires n–2 divisor checks rather than approximately √n–1. Composite numbers stop at their first divisor, so their counts depend on the smallest divisor. Trial division is O(√n) time and O(1) extra space. The Sieve of Eratosthenes precomputes primes through N in O(N log log N)

time and O(N) space; each later primality query is O(1).

Q6 – Loops That Must Terminate: Loan Repayment Schedule

1. Calculation and loop design

A while loop is appropriate because the number of months is not known in advance and the loop should continue while a positive balance remains. Use a small currency tolerance or integer paise in a real billing system; do not compare floating-point balance with != 0. The final repayment must be limited to the balance after interest. If repayment is less than or equal to the monthly interest, the balance will not decrease (and may grow), so the program must detect this.

```
#include <stdio.h>
int main(void) {
    double balance=10000.00, rate=0.02, payment=2000.00;
    double total_interest=0.0;
    int month=0;
    const int LIMIT=360;

    while (balance > 0.005 && month < LIMIT) {
        double interest=balance*rate;
        double due=balance+interest;
        double paid=(payment < due) ? payment : due;
        if (payment <= interest && paid < due) {
            printf("Repayment does not reduce the loan.\n");
            return 0;
        }
        balance=due-paid;
        total_interest += interest;
        month++;
        if (balance < 0.005) balance=0.0;
        printf("Month %d: interest=%.2f paid=%.2f balance=%.2f\n",
               month,interest,paid,balance);
    }
    if (balance > 0.005)
        printf("Safety limit reached; outstanding balance=%.2f\n",balance);
    else
        printf("Months=%d Total interest=%.2f\n",month,total_interest);
    return 0;
}
```

2. Hand-verified schedule

Month	Opening balance	Interest (2%)	Due	Payment	Closing balance
1	10,000.00	200.00	10,200.00	2,000.00	8,200.00
2	8,200.00	164.00	8,364.00	2,000.00	6,364.00
3	6,364.00	127.28	6,491.28	2,000.00	4,491.28
4	4,491.28	89.83	4,581.11	2,000.00	2,581.11
5	2,581.11	51.62	2,632.73	2,000.00	632.73
6	632.73	12.65	645.38	645.38	0.00

Total interest = Rs 645.38 (rounded to paise at each displayed month; using full precision internally may differ by a few paise). The loan is repaid in 6 months. The safety limit prevents an accidental infinite loop; if reached, report that the loan is not fully repaid and show the remaining balance.

If repayment decreases toward the monthly interest, the balance decreases more slowly and iteration count rises. If repayment is no greater than interest, repayment does not amortize the principal. The direct logarithmic formula can estimate the number of periods in O(1) time and O(1) space, but assumes a fixed payment and rate and needs careful rounding/last-payment adjustment. Monthly simulation is O(m) for m months, uses O(1) space, and gives a transparent schedule.

Q7 – Diagnose, Redesign and Validate

Selected program: Q5 prime-number checker

A. Diagnose the bottleneck. The original trial-division approach tests every possible divisor from 2 through n−1. For a large prime such as 1,000,003, that would require 1,000,001 remainder checks. Most checks are unnecessary because a composite factor pair must contain at least one factor no greater than √n.

B. Measurable objective. Reduce the number of divisibility tests for a large prime from approximately 1,000,001 to 999, while preserving the correct prime/non-prime result.

C. Revised solution. Stop at the square root and skip even divisors after testing 2. This further reduces checks. No array or extra memory is required; only the loop variable and counter are used.

```
int is_prime_fast(unsigned long n, unsigned long *checks) {
    *checks=0;
    if (n<2) return 0;
    if (n==2) return 1;
    if (n%2==0) { (*checks)++; return 0; }
    for (unsigned long i=3; i<=n/i; i+=2) {
        (*checks)++;
        if (n%i==0) return 0;
    }
    return 1;
}
```

The revised algorithm checks 2 once for an even input, then checks only odd divisors 3, 5, 7, … up to √n. It uses O(√n) time in the worst case (about half as many trial candidates as the square-root version) and O(1) extra space. For 1,000,003, the odd candidates run from 3 to 999, so there are 499 odd-divisor checks after the special handling of 2. The trade-off is a small amount of extra branching, but correctness is unchanged.

D. Validation

Test at least: n=–1 and 0 (not prime), 1 (not prime), 2 and 3 (prime), 4 (not prime), 9 (not prime), 97 (prime), 100 (not prime), and 1,000,003 (prime). Compare outputs from the original, square-root and odd-only versions. All three must agree on primality; the redesigned versions should perform fewer checks for large prime inputs.

Submission note

The explanations and sample data above provide a complete worked solution based on the questions in the supplied assignment. For Q2, the passenger distribution is an explicit example assumption because the PDF does not provide one. For Q6, the schedule uses monthly interest of 2% and a fixed Rs 2,000 repayment, as suggested in the assignment.